

site_builder/ 完整 Function Inventory

目的：作為後續依「功能」拆分 site_builder/ 程式碼時的判斷依據。

涵蓋每個檔案內的所有 function（用途、參數、回傳內容、副作用）、所有常數，

以及會直接影響拆分決策的「跨檔案重複 / 職責混雜 / 死碼」觀察。

產出時間基準：2026-07-01，逐行讀完 site_builder/ 全部 9 個檔案後整理（2026-07-01 第二版：納入新增的 wrc_plus.py 與 wrc+/TJBat+ 功能上線後對 builder.py / statcast.py / jinja_env.py 的後續修改）。

site_builder/	
__init__.py	0 行（空檔，純套件標記）
api.py	419 行 — MLB Stats API client（純 I/O 邊界層）
levels.py	149 行 — 等級（MLB/AAA/AA/...）唯一真實來源
jinja_env.py	183 行 — Jinja2 filters / globals / URL 工廠
helpers.py	687 行 — 共用工具 + 數據聚合公式（職責最雜的小檔）
sync.py	1437 行 — DB schema + 兩條平行的抓取/寫入 pipeline（最大檔）
statcast.py	1399 行 — 逐球資料抽取與進階指標計算
wrc_plus.py	229 行 — TJBat+（wrc+）計算（新增模組，向 tstats.ca 抓 park factors / league constants）
builder.py	1217 行 — SQLite → Jinja2 → HTML（職責最雜的大檔）

0. 模組依賴關係（import 方向）

api.py	→ levels.py
jinja_env.py	→ levels.py
helpers.py	→ levels.py（re-export，供 sync.py / builder.py 用 from .helpers import）
sync.py	→ api.py, levels.py, helpers.py, statcast.py
wrc_plus.py	→ statcast.py（僅 WOBAB_WEIGHTS）+ 外部套件 requests / bs4（直接打 tstats.ca）
builder.py	→ helpers.py, jinja_env.py, statcast.py（僅 summarize_pitch_for_display）, wrc_plus.py, api.py（僅 parse_roster_from_file）
statcast.py	→ （無內部依賴，僅 math / typing）
levels.py	→ （無內部依賴，套件中唯一的「純常數註冊表」模組）

wrc_plus.py 是套件中唯一會在 build time 對外部網站（tstats.ca）發 HTTP request 的模組（api.py 只打 statsapi.mlb.com）。它的計算結果刻意不寫回 SQLite——annotate_wrc_plus 直接原地修改 _load_player_bundle 回傳的 Obj 列，每次 build 都重新抓取重新算，與套件內其他 derived stat（_compute_advanced_stats 等）的「不持久化，每次重算」慣例一致。

levels.py 是全套件中唯一已經做到「單一事實來源」的模組，可作為其他常數表（roster 狀態碼、球種代碼）重構時的範本。

helpers.py 對 levels.py 的 6 個函式做了 re-export

（is_mlb, level_display, level_rank, resolve_tier, sport_id_to_code, tier_keys_ordered，helpers.py:15-22），且這個 re-export 目前仍在被使用（sync.py、builder.py 都是寫 from site_builder.helpers import (...) 而非 from .levels import）。拆分 helpers.py 時必須保留這個轉接，或同步改掉兩個呼叫端的 import。

1. site_builder/__init__.py

空檔，僅作為套件標記。無 function、無常數。

2. site_builder/api.py — MLB Stats API client

純 HTTP I/O 邊界層，每個函式對應 1~多個 MLB Stats API endpoint，回傳結構化 dict/list，不寫入資料庫、不做業務判斷（roster 狀態分類等留給 helpers.py）。

常數

常數	值	說明
BASE_URL	https://statsapi.mlb.com/api/v1	全檔案唯一 base URL
TIMEOUT	15 （秒）	預設 request timeout ；不是所有請求都用這個值（見下方備註）

Functions

`get_player_profile(mlb_id: int) -> dict` (api.py:20-121)

- Endpoint: `/people/{mlb_id}?hydrate=transactions,rosterEntries,currentTeam`，若有 `team_id` 會再打一次 `/teams/{team_id}` 取得 sportId 換算等級。
- 回傳 dict 欄位：
`mlb_id, full_name, position, height, weight, birth_date, birth_city, birth_country, is_active, bat_side, pitch_hand, latest_transa`。
- 查無球員時回傳 `{}`。
- 副作用：最多 2 次 HTTP request；第二次 (team level) 失敗只 log warning，`current_team_level` 留空字串，不丟例外。
- 依賴 `levels.sport_id_to_code`。

`get_player_stats(mlb_id) -> list` (api.py:124-152)

- 依序打 MLB `yearByYear`（`group=batting,pitching,fielding`）與 MiLB `yearByYear`（`leagueListId=milb_all`），合併兩者 stats list。
- ⚠️ 錯誤處理不對稱：MLB 呼叫包在 `try/except` 裡 (api.py:135-141)，但 MiLB 呼叫沒有 (api.py:144-150)，MiLB API 失敗時例外會直接往外拋，與其他函式的「兩端點都吞例外」慣例不一致。

`get_player_advanced_stats(mlb_id, years=None) -> list` (api.py:155-195)

- 對每個年份（或單一 `[None]`）各打一次 MLB + MiLB 的 `seasonAdvanced`（`group=batting,pitching`），兩端點都包 `try/except`。
- HTTP 呼叫數 = `len(years) * 2`。

`get_game_logs(mlb_id, season) -> list` (api.py:198-229)

- MLB + MiLB 的 `gameLog`（`group=batting,pitching`），兩端點都包 `try/except`，合併回傳。
- 註解明確說明：永遠兩個端點都打，是為了 shuttle 球員（MLB↔MiLB 來回）能拿到完整 game log。

`get_next_game(team_id) -> Optional[dict]` (api.py:232-290)

- `/schedule` 7 天窗口，`sportId=1,11,12,13,14,15,16`（涵蓋 MLB 到 ROK）。
- 回傳第一筆 `status.abstractGameState == "Preview"` 的比賽：`date, opponent, is_home, venue, game_time(UTC+8 格式化字串), status`。
- `team_id` 為空、HTTP 失敗、或無未來賽事時回傳 `None`。

`get_game_play_by_play(game_pk) -> dict` (api.py:293-306)

- `https://statsapi.mlb.com/api/v1.1/game/{game_pk}/feed/live` 完整 live-feed JSON。
- timeout 寫死 30（不是 TIMEOUT 常數）。失敗回傳 `{}`。
- 文件註明呼叫者要自己走 `liveData.plays.allPlays` 抽逐球資料（實際由 `statcast.extract_pitch_logs` 做）。

`sport_obj_to_abbr(sport: dict) -> str` (api.py:309-320)

- 把 live-feed 裡的 sport 物件轉成等級縮寫；先試 `sport_id_to_code`，沒命中再 fallback `sport_name_to_code`。空 dict 回傳 `""`。

`get_game_sport_level(game_pk) -> str` (api.py:323-349)

- 用 `fields=` 過濾參數打一次輕量 live-feed 請求，只為了拿 `gameData.teams.home.sport`。timeout 寫死 15（與 TIMEOUT 常數同值但沒引用常數，是另一個獨立字面量）。失敗回傳 `""`。

`get_player_sabermetrics(mlb_id, years=None) -> list` (api.py:352-371)

- 僅 MLB 端點 `stats=sabermetrics&group=pitching,batting`（FIP/xFIP/WAR 來源）。MiLB 沒有這個 stat type。

`get_player_expected_stats(mlb_id, years=None, group="pitching") -> list` (api.py:374-407)

- 僅 MLB 端點 `stats=expectedStatistics` 。docstring 明確說明 MiLB 端點一律回傳 0.0 (API 限制)，所以根本不打。
- 注意 API 欄位名稱是 `avg/slg/woba/wobaCon` (無 x 前綴)，呼叫端要自己改名成 `xba/xslg/xwoba/xwobacon` (實際在 `sync.py:1174-1177` 做這個轉換)。

`parse_roster_from_file(filepath) -> list` (api.py:410-419)

- 讀 roster JSON 檔，回傳 `data.get("players", [])`；讀檔/解析失敗回傳 `[]` (log error)。
- 被 `sync.py` 與 `builder.py` 兩邊各自獨立呼叫 (`builder.py` 內是 local import，見 `builder.py:814`)。

拆分相關觀察

- 已經是職責單一、純 I/O 的好模組，多數函式不需要大改。
- `timeout` 數值不一致 (15 / 30 / 字面量 15) 值得在拆分時順手統一成具名常數。
- `get_player_stats` 的 try/except 不對稱是需要修的小 bug，不只是拆分問題。
- 若要拆，可依資源類型切：`people.py` (profile/roster)、`stats.py` (yearByYear/advanced/sabermetrics/expected)、`schedule.py` (next_game)、`playbyplay.py` (live feed 兩個函式)。

3. site_builder/levels.py — 等級唯一真實來源

全套件「示範級」模組：所有 MLB/MiLB 等級邏輯 (sportId 對應、歷史拼法別名、階層 rank、年代相關顯示字串) 集中於此，檔頭註解明確聲明「其他模組不得自建等級常數表」。

常數

常數	說明
<code>_MODERN_FROM_YEAR = 2021</code>	2021 年起採用重組後新名稱 (2020 因疫情無 MiLB 球季，分界乾淨)
<code>_SENTINELS = frozenset({"_combined", "_all", ""})</code>	前端篩選用的哨兵值，永遠不當作真實等級處理
<code>Tier</code> (frozen dataclass)	欄位： <code>key</code> , <code>rank</code> , <code>sport_ids(tuple)</code> , <code>modern(Optional[str])</code> , <code>legacy(str)</code> , <code>aliases(tuple)</code>
<code>TIERS</code>	9 個 <code>Tier</code> ：MLB(0), AAA(1), AA(2), A+(3), A(4), A-(5,已廢除/modern=None), ROK(6), WIN(7), N
<code>_UNKNOWN_RANK = 50</code>	未知等級的 fallback rank (低於所有真實等級，高於 Minors 聚合的 99)
<code>_BY_ALIAS / _BY_SPORT_ID / _BY_NAME</code>	由 <code>TIERS</code> 衍生的 3 個查找 dict

Functions

- `resolve_tier(raw) -> Optional[Tier]`：任意原始拼法 (含歷史拼法) → `Tier`。
- `level_rank(raw) -> int`：排序用 rank，未知等級回傳 50。
- `level_display(raw, year) -> Optional[str]`：依 `year` 決定用 2021+ 現代名稱還是 2020- 期間名稱；哨兵值與未知值原樣傳回。
- `is_mlb(raw) -> bool`：是否為 MLB tier (用於 hero badge 特殊樣式)。
- `sport_id_to_code(sport_id) -> str`：sync 時把 currentTeam/game-log 的 `sportId` 轉成儲存用等級碼；`sportId 15` (已廢除的 short season) fallback 用 `legacy` 名稱，確保不回空字串。
- `sport_name_to_code(name) -> str`：同上但用官方 `sport name` 查 (`sport_id_to_code` 的備援)。
- `tier_keys_ordered() -> list[str]`：依 rank 排序的 tier key 清單，給 SQL CASE 排序用。

拆分相關觀察

- 不需要拆，可以原樣保留，是其他常數表重構的參考範本。
- `sync.py:653-657` 自己用 `for t in TIERS for alias in t.aliases` 手刻一段 SQL CASE 字串，而非呼叫 `levels.py` 提供的工具——這是「迭代 TIERS 別名」這個 pattern 的小重複，可考慮在 `levels.py` 增加一個 `level_rank_sql_case(column_name)` 之類的工具函式收斂掉。

4. site_builder/jinja_env.py — Jinja2 環境設定

常數

常數	說明
<code>_PROJECT_ROOT</code> , <code>_TEMPLATE_DIR</code>	路徑常數，定位 <code>src/templates</code>
<code>HEADSHOT_CDN_TEMPLATE_MLB</code>	MLB 大頭照 CDN URL 樣板 (Cloudinary , <code>/headshot/67/current</code>)
<code>HEADSHOT_CDN_TEMPLATE_MILB</code>	MiLB 大頭照 CDN URL 樣板 (<code>/headshot/milb/current</code>)

Functions (Filters)

- `floatformat(value, digits=2) -> str` : 固定小數位數，`None` 回傳 `"-"` 。
- `default_if_none(value, fallback="-") -> Any` : `None` 時回傳 `fallback` 。
- `num_dash(value) -> Any` : `None` / `""` 回傳 `"-"` , 否則原樣回傳。
- `_json_html_safe(s) -> str` : 把 `</` 轉成 `</` , 防止內嵌 `<script>` 被提前關閉。
- `tojson_safe(value) -> Markup` : `json.dumps(ensure_ascii=False)` + HTML-safe, 用於嵌入 `<script>` 。
- `jsonld(value) -> Markup` : 同上但用緊湊分隔符 (`separators=(",",":")`) , 給 JSON-LD 結構化資料用。
- `pct_fmt(value, digits=1) -> str` : 小數 (如 0.345) → 百分比字串 ("34.5%") , 用 `Decimal` + `ROUND_HALF_UP` 精確四捨五入 , `None` 回傳 `"-"` 。

Functions (URL 工廠 / Globals)

- `headshot_cdn_urls(mlb_id, latest_level_is_mlb) -> (primary, secondary)` : 依「球員最近一個有實際出賽的球季所在等級」決定先試 MLB 還是 MiLB CDN (因為兩個圖庫互斥, 球員一生只會有其中一個被更新過) 。
- `_make_url_helpers(base_url) -> (player_url, retired_player_url, static_url)` : closure 工廠, 產生 3 個 URL 產生函式。
- `_make_absolute_url(site_origin, base_url) -> (site_root, absolute_url)` : closure 工廠, 產生絕對 URL 轉換函式 (給 sitemap/結構化資料用) 。
- `create_jinja_env(template_dir=None, base_url="/", site_origin="https://tingruih.github.io") -> Environment` : 環境工廠, 正規化 `base_url` (強制前後加 `/`) , 註冊上述所有 filters 與 globals (`is_mlb, player_url, retired_player_url, static_url, headshot_cdn_urls, absolute_url, base_url, site_url, site_origin`) 。

拆分相關觀察

- 已用 `#` — Custom Filters — / `#` — URL Factories — / `#` — Environment Factory — 三段分隔, 邊界清楚, 不太需要拆。
- `headshot_cdn_urls` 同時被 `env.globals` 註冊 (給 `template` 用) 也被 `builder.py:671` 直接 `import` 呼叫 (給結構化資料的 `image` 欄位用) —— 是少數「rendering 工具函式被資料層直接呼叫」的案例, 拆分時注意這個雙重用途。

5. site_builder/helpers.py — 共用工具 + 數據聚合公式

職責最雜的小檔：實際上揉合了 5 種不同性質的東西，是除了 `builder.py` 外最值得拆分的候選。

常數

常數	說明
<code>DEFAULT_SEASON_YEAR</code>	<code>int(os.environ.get("DEFAULT_SEASON_YEAR", "2026"))</code> 。 已用 <b grep 確認： <code>site_builder</code> 內部完全沒人用，唯一呼叫端是 <code>build.py:38</code> (CLI 預設年份)。放在 <code>helpers.py</code> 純粹是歷史位置，邏輯上更接近 <code>build.py</code> 或一個 <code>config.py</code> 。
<code>Obj(dict)</code> class	支援屬性存取的 <code>dict</code> (<code>obj.foo ↔ obj["foo"]</code>) , 是 <code>builder.py</code> 全面使用的資料載體； <code>sync.py</code> 內部仍用原生 <code>dict</code> , 沒有用到 <code>Obj</code> 。
<code>ROSTER_INJURED_CODES</code>	<code>{D7,D10,D15,D60,ILF,RA}</code>
<code>ROSTER_RESTRICTED_CODES</code>	<code>{SU,RES,BRV,FME,RST,IN,PL,MIL,ADM,TI}</code>

常數	說明
ROSTER_OTHER_CODES	{DES}
ROSTER_INACTIVE_CODES	{RL,RET,VL}
_COUNTING_FIELDS	~50 個欄位名稱清單，career/season-combined 聚合時要 sum 的計數型欄位（打擊/投球/進階各自一段）
_NATIONAL_TEAM_KEYWORD = "chinese taipei"	判斷交易紀錄是否為國家隊徵召（不計入「仍在體系內活躍」判斷）
_HEIGHT_RE	身高字串（ 6'2" ）解析用 regex

Functions

Roster 狀態分類

- categorize_roster_status(code, is_active_entry, player_is_active) -> str：回傳 "active"/"injured"/"restricted"/"inactive"/"other"。是 sync.py、builder.py 共用的單一分類入口。

安全型別轉換

- safe_float(value, default=None)、safe_int(value, default=None)：轉型失敗回傳 default，不丟例外。

JSON 工具

- loads_json(text, default)：已是 dict/list 直接回傳，否則 json.loads，失敗回傳 default。
- loads_json_dict(text) -> dict、loads_json_list(text) -> list：上面的型別保證版本。
- dumps_json(value) -> str：ensure_ascii=False, separators=(",", ":") 緊湊序列化。

日期 / 單位工具

- parse_date(text) -> Optional[date]：取前 10 字元做 ISO 解析，失敗回傳 None。
- ip_to_outs(ip_value) -> int：棒球小數記法 IP（7.2 = 7%局） -> outs 數。
- outs_to_ip(outs) -> Optional[float]：反運算，0 outs 回傳 None。
- height_to_cm(height_str) -> Optional[float]：6'2" -> cm。
- lbs_to_kg(weight_lbs) -> Optional[float]。
- calc_obp(hits, bb, hbp, ab, sac_flies) -> Optional[float]：標準 OBP 公式，分母為 0 回傳 None。

等級 / 出賽判斷

- highest_level_row(stats) -> Optional[stat row]：用 level_rank 找出最高等級那一筆（優先選有實際出賽的列）。
- highest_level(stats) -> Optional[str]：包裝上面，只回傳 tier key 字串。
- _is_national_team_tx(tx) -> bool：交易描述含「chinese taipei」即為國家隊徵召。
- is_active_player(player, stats, year) -> bool：驅動首頁 / Retired 頁分流的核心業務邏輯（被 builder.py:886 呼叫）。判斷規則：當年有 season_stats 列 或 當年有「非國家隊徵召」的交易紀錄，才算 active。
- has_appearance(stat) -> bool：gp/pa/ab/bf > 0 或 ip_to_outs(ip) > 0 任一成立即為有實際出賽。

聚合核心

- _sum_counting(stats, result)：對 _COUNTING_FIELDS 逐欄加總，**原地修改** result；全部為 None 則該欄結果也是 None（而非 0）。
- _compute_rate_stats(agg)：原地補上 avg/obp/slg/ops（需要 ab>0）與 era/whip（需要 ip_to_outs 轉真實局數 >0）。註解特別強調 IP 必須先轉 outs 再除 3，否則 ERA/WHIP 會算錯。
- _aggregate_stats(stats) -> Obj：_sum_counting + IP 加總轉換 + _compute_rate_stats 的組合，是 compute_career / compute_season_combined 共用核心。
- compute_career(stats, level_filter=None) -> Optional[Obj]：level_filter 可為 "mlb"/"milb"/None；額外算出 teams_display（"AAA TeamX / MLB TeamY"）與 years_range（"2019-2024"）。
- compute_season_combined(stats, year) -> Optional[Obj]：單一年度跨隊伍合併（同球季多隊）。

進階衍生指標（單一巨型函式）

- _fmt_avg(value) -> Optional[str]：去掉開頭 0 的棒球式格式（.333 而非 0.333，1.000 例外保留）。
- _compute_advanced_stats(s)（helpers.py:418-632，約 **215 行，全檔最大函式**）：原地補上 ~30 個衍生欄位，只在欄位目前是 None 時才填（API 已提供的值絕不覆寫）。內容同時涵蓋打者與投手兩種公式族：

- 打者：p_per_pa, xbh, iso, babip, ab_per_hr, go_ao, sb_pct, k_pct, bb_pct
- 投手：
- 手：
- 這是全套件中最強烈的「該拆分」訊號：一個函式做兩種完全不同球員角色的公式，自然邊界就是檔案內已用空行+註解分隔的 BATTER fields / PITCHER fields 兩段 ([helpers.py:432, 507](#))。
- annotate_computed_stats(all_stats) -> list：對每筆 stat 加 .np (pitches 別名) + 呼叫 _compute_advanced_stats，原地修改後回傳同一個 list。
- compute_year_groups(all_stats) -> list[dict]：依年份分組 (新到舊)，每組 {year, summary(Obj, 重新算 rate stats), rows(list[Obj], 依 level_order 排序), multi(bool, 是否跨隊)}。

拆分相關觀察

這個檔案實際上是 5 種職責的合集，且檔案內已經用 # — 分隔標題自然劃出邊界：

1. 泛用安全轉換 / JSON 工具 (→ 可獨立成真正的 utils.py)
2. roster 狀態分類常數與函式 (→ 與球員 profile 相關，邏輯上更靠近 api.py / sync.py 的球員資料層)
3. 單位轉換 (身高體重，純展示層關心的東西)
4. 統計聚合公式 (最大宗、最具領域知識：_COUNTING_FIELDS、_compute_advanced_stats、_aggregate_stats 家族、compute_career / compute_season_combined / compute_year_groups)
5. Obj 資料載體類別本身

若要拆，_compute_advanced_stats 內部「打者公式 / 投手公式」的邊界已經現成存在，是最容易、效益最高的第一刀。

6. site_builder/sync.py — DB schema + 抓取/寫入 pipeline (全套件最大檔)

本質上是兩條幾乎獨立的 pipeline 被放進同一個檔案：

- Pipeline A：球員 profile + yearByYear 季成績 + game log (sync_database / update_database)
- Pipeline B：Statcast 逐球資料抓取與聚合 (sync_statcast)

兩者只共用 MAX_WORKERS、parse_roster_from_file、以及「自己開 sqlite 連線」這個 pattern，業務邏輯完全不重疊。

常數

常數	說明
MAX_WORKERS = 10	被兩條完全不同性質的 pipeline 共用的執行緒池大小 (Pipeline A 是「打 N 個球員的 profile/stats API」，Pipeline B 是「打 N 場比賽的 live-feed API」) ——兩者最佳並行數不一定相同，目前用同一個數字。

Functions

DB Schema (_init_db, sync.py:52-145)

- 建立 4 張表：players、season_stats (UNIQUE(player_mlb_id, year, team_name))、game_logs (UNIQUE(player_mlb_id, game_id))、playbyplay_processed (game_pk 主鍵) + 2 個索引。
- 緊接著 5 段 ALTER TABLE ... ADD COLUMN + try/except sqlite3.OperationalError: pass 的**就地遷移** (pitches_json, sport_level, roster_status_code, roster_is_active, hit_coord_checked)。每次要加新欄位都要照這個 pattern 手寫一段，且散落在同一個函式裡，沒有獨立的 migrations 機制。

Season row 讀寫

- _load_season_row(cur, mlb_id, year, team_name) -> dict：{league_name, sport_level, stat_json(dict), fielding_json(list)}，查無回傳全空殼。
- _save_season_row(cur, ..., stat_json, fielding_json)：INSERT ... ON CONFLICT DO UPDATE upsert。
- _players_with_existing_stats(conn) -> set[int]：已有 season_stats 列的 mlb_id 集合 (判斷「是否首次同步」用)。

- `_warn_orphaned_players(conn, roster_ids)`：純列印診斷訊息（DB 裡有、roster.json 裡沒有的球員），並印出建議的清理 SQL，無回傳值。
- `_is_first_sync(mlb_id, synced_ids) -> bool`。

API → DB 欄位映射

- `_apply_yearbyyear_fields(stat_doc, group_name, stat)`：原地修改 `stat_doc`，pitching ~45 個欄位映射、hitting ~25 個欄位映射（MLB API 駝峰命名 → 內部 snake_case）。
- `_apply_advanced_fields(stat_doc, group_name, stat)`：原地修改，seasonAdvanced 的較小欄位集（roe, wo, gidpo, xbh, babip, pitches_per_pa / qs, bqr, bqr_s, p_gidpo, run_support, rs_per_9, p_babip 等）。

Pipeline A：平行抓取（profile/season）

- `_fetch_player_data(pconf, year, fetch_all_years=True) -> Optional[dict]`（sync.py:398-508）：純抓取，不寫 DB，**thread-safe**。呼叫 `get_player_profile` → `categorize_roster_status` →（若 inactive 且非首次同步，提早回傳精簡 bundle，跳過重量級抓取）→ `get_player_stats` → `get_player_advanced_stats`（年份依模式決定全部還是只抓當年）→ 逐年 `get_game_logs` → `get_next_game`（inactive 球員跳過，因為 `team_id` 會是舊球隊）。回傳 bundle
dict：{pconf, profile, status_category, stats_groups, adv_groups, log_groups, next_game, years_with_data}；無 profile 回傳 None。
- `_write_player_to_db(conn, bundle, year)`（sync.py:511-725）：bundle 的 DB 寫入端。Upsert players 列 → 遍歷 stats_groups（yearByYear）寫 season_stats + fielding_json（fielding 的 gp 不可覆蓋總 gp，因為是逐守位資料）→ 遍歷 adv_groups merge 進既有 season_stats → 更新 players.level/team（有 profile 直接用，否則用 TIERS 動態組 SQL CASE 從最新 season_stats 推導）→ 寫 game_logs upsert（sport_level 用 COALESCE-like 邏輯保留舊值）→ 寫 next_game_json 快照。最後 `conn.commit()`。

Pipeline A：orchestration

- `_run_pipeline(db_path, roster_file, year, only_player=None, fetch_all_years=True, mode_label="Sync")`（sync.py:731-839）：Phase 1 用 `ThreadPoolExecutor(MAX_WORKERS)` 平行呼叫 `_fetch_player_data`；Phase 2 序列呼叫 `_write_player_to_db`（SQLite 寫入序列化）。額外處理：快取為「永久離隊」的球員直接跳過整輪抓取（除非 `--player` 指定或首次同步）；結束後呼叫 `_warn_orphaned_players`。進度回報全用 `print()`，例外才用 `logger.exception`。
- `sync_database(db_path, roster_file, year, only_player=None)`：薄封裝，`fetch_all_years=True`。
- `update_database(db_path, roster_file, year, only_player=None)`：薄封裝，`fetch_all_years=False`（只抓當年 game log，較快）。

Pipeline B：Statcast 同步

- `_build_roster_map(roster_file) -> dict[mlb_id, pconf]`。
- `_fetch_and_extract_game(game_pk, players_in_game) -> (dict[mlb_id, pitches], sport_level)`：一場比賽只打一次 **live-feed**，攤給該場所所有相關球員抽逐球資料；找不到逐球資料時會試對調角色（pitcher↔batter，處理雙向球員 / roster 配置錯誤）。
- `_pitches_need_hit_coord_backfill(pitches) -> bool`：偵測舊資料缺 hit_coord_x/y 的場次，觸發補抓。
- `_load_all_pitches_for_player(cur, mlb_id) -> dict[(year, sport_level), pitches]`：合併該球員所有已快取的逐球資料；sport_level 為空時嘗試從 season_stats 推導（單一等級才能無歧義推導，否則歸到（year, ""）留給呼叫端處理）。
- `_merge_statcast_into_season(cur, mlb_id, year, position, statcast_data, sport_level="", sabermetrics=None, expected_stats=None)`（sync.py:1003-1117）：把計算好的 statcast/sabermetrics/expected/FIP/xWPCT 寫回對應的 season_stats 列。投手 MiLB 用 `statcast.compute_fip`（搭配 `FIP_CONSTANTS`）算 FIP；投手 MLB 直接用 `sabermetrics` 端點自帶的 `fip/xfip/war`。用 `saber_written` 旗標避免轉隊球員的 `wRC+/WAR` 被重複寫進每一筆 MLB 列（同年多隊只寫第一筆）。
- `_compute_player_statcast_bundle(mlb_id, db_path, position) -> (mlb_id, Optional[dict])`（sync.py:1120-1204）：**平行 worker**，自己開唯讀 sqlite 連線，載入逐球資料 → 視 `has_mlb_stats` 決定要不要打 `sabermetrics` 端點 → 打 `expectedStats` 端點 → 對每個（year, level）呼叫 `statcast.compute_pitcher_statcast / compute_batter_statcast`。回傳 `dict[(year, level)] = {statcast, sabermetrics, expected_stats}`。
- `sync_statcast(db_path, roster_file, year, only_player=None)`（sync.py:1207-1437）：總入口，4 個 phase：
 - **Phase 0**：補齊歷史 game_logs 缺失的 sport_level（平行呼叫 `get_game_sport_level`）。
 - **Phase 1**：找出所有「需要抓逐球資料」的（game_pk, [(mlb_id, position)]）（`pitches_json` 為空，或需要 hit-coord 補抓）。
 - **Phase 2**：平行 `_fetch_and_extract_game`。
 - **Phase 3**：序列把 `pitches_json` 寫回 game_logs，標記 `playbyplay_processed`（無逐球資料的場次寫 JSON null 而非 "[]"，避免被誤判為「尚未抓取」造成無限重抓迴圈）。
 - **Phase 4**：平行 `_compute_player_statcast_bundle` + 序列 `_merge_statcast_into_season`（每位球員各自 `conn.commit()`）。

拆分相關觀察

全套件最大檔，內部至少藏了三種可獨立的職責：

- 1. **DB schema / migrations** (`_init_db`) ——目前用「ALTER TABLE + 吞例外」手刻遷移，沒有版本化機制，新增欄位的成本與風險會隨表增多而上升。
- 2. **Pipeline A (profile/season 同步)** —— `_fetch_player_data` / `_write_player_to_db` / `_run_pipeline` / `sync_database` / `update_database` 。
- 3. **Pipeline B (statcast 同步)** —— `_build_roster_map` 以下到 `sync_statcast` ，完全是另一套抓取對象（比賽而非球員）、另一套並行單位（`game_pk` 而非 `mlb_id`）。

`_apply_yearbyyear_fields` / `_apply_advanced_fields` 這兩個「API 欄位名 → 內部欄位名」的大型映射表，知識上更貼近 `api.py`（描述 API 回應形狀）而非 `sync.py`（負責 orchestration），拆分時可以考慮獨立成 `field_mapping.py` 。

`print()` 做進度回報、`logger` 只記例外，這個不一致的紀錄方式在拆成多檔後需要決定要不要統一（例如共用一個 progress reporter）。

7. site_builder/statcast.py — 逐球資料抽取與進階指標

檔案內部已用區塊標題自我組織（EXTRACTION / CLASSIFICATION HELPERS / SHARED AGGREGATION HELPERS / PITCHER AGGREGATION / BATTER AGGREGATION / FIP / xWPCT / PITCH-LOG DISPLAY HELPERS），是內部結構第二好的檔案（僅次於 `levels.py`）。

常數

常數	
SWING_CODES	{S,W,F,T,M,X,D,E,Z,L,Q}，揮棒判定碼（含
WHIFF_CODES	{S,W,T,M,Q}，揮空判定碼
CALLED_STRIKE_CODES	{C}
WOBA_WEIGHTS	TJStats 固定 wOBA 線性權重：walk 0.689, hbp 0.720, single tjstats.ca ），所有等級/年份共用同一組權重
WOBA_EVENT_MAP	API eventType 字串 → wOBA key 對照
FIP_CONSTANTS	{(sport_level, year): constant}，目前只
LEAGUE_RA9	{sport_level: 近似聯盟 RA/9}，xWPCT 公
_NON_PA_EVENTS	跑壘相關事件（盜壘刺殺、牽制出局等），需從
_BAT_SIDE_SPLITS	(("all", "全部"), ("L", "左打"), ("R", "右打"))
_COUNT_USAGE_BUCKETS	5 個 dict (key, label, counts_label, counts(形狀不同但語意重複
_PLINKO_COUNTS	12 個球數 tuple (0,0)~(3,2) △ 與 builder.p
_PLINKO_EDGES	17 組合法球數轉移 △ 與 builder.py 重複
_BATTER_PLINKO_SPLITS / _PITCHER_PLINKO_SPLITS	左右打/投對照標籤
_BATTER_PLINKO_SKIP_TYPES	{EP, FA}
_GB/_LD/_FB/_PU_TRAJECTORIES、_AIR_TRAJECTORIES	打點軌跡分類集合
_BATTED_BALL_RATE_DIGITS = 6	打點比率小數位數
_GAMEDAY_HOME_X/_Y、_GAMEDAY_SPRAY_CORRECTION、_GAMEDAY_LEFT/RIGHT_FIELD_THRESHOLD_DEG	Gameday 噴射圖座標系/角度公式常數（含完
_HIT_LOCATION_ZONE	守位代碼 → LF/CF/RF 粗略分區（座標缺失時的

Functions

- `get_woba_weights(year=None)` -> dict：固定回傳 WOBA_WEIGHTS，year 參數僅為保留舊呼叫介面相容性。△ 已用 **grep** 確認全 repo 零呼叫端 ——wOBA 權重改成年份無關的固定表後（見上方 WOBA_WEIGHTS 常數說明），這個函式整個變成死碼，拆分時可直接刪除。

EXTRACTION

- `extract_pitch_logs(game_data, player_id, role) -> list[dict]` (statcast.py:204-324)：走訪 `liveData.plays.allPlays`，篩出 `role="pitcher"/"batter"` 對應 `player_id` 的每一球，輸出 ~35 個欄位的逐球 dict (球種、結果碼、球路物理量、球數狀態、PA 終局事件歸屬)。這個函式定義的 **dict shape 就是 `pitches_json` 的 `schema`**，下游所有計算函式都依賴這個形狀。
- `_ensure_pre_strikes(pitches) -> None`：原地補上舊快取資料缺的 `pre_balls/pre_strikes` (依 `game_pk` 邊界重置)。

CLASSIFICATION HELPERS

- `_is_swing/_is_whiff/_is_called_strike/_is_in_zone/_is_out_of_zone(p) -> bool`
- `_is_barrel(ev, la) -> bool`：Statcast barrel 定義 (98mph 起算，角度窗隨 EV 線性放寬，含註解推導的錨點)。
- `_is_sweet_spot(la) -> bool`：8°~32°。
- `_ratio(num, den, digits=3) -> Optional[float]` △ 與 **builder.py** 的 `_ratio` 重複但預設精度不同 (**builder.py** 寫死 4 位)
- `_mean(values) / _mean_round(values, digits=1)`
- `_float_or_none(value) -> Optional[float]`：含 `math.isfinite` 檢查。
- `_is_unknown_pitch_type(pitch_type, pitch_name=None) -> bool` △ 與 **builder.py** 完全相同實作，逐字重複
- `_filter_known_pitch_events(pitches) -> list[dict]`
- `_pre_count_tuple(p) / _post_count_tuple(p) -> Optional[tuple[int,int]]`
- `_count_label(count) -> str`："0-0" 格式。
- `_empty_plinko_nodes() / _empty_plinko_edges() -> list[dict]`：零值骨架。
- `_compute_pitch_plinko(pitches, *, split_field, split_specs, skip_types=None) -> dict`：泛用的「Pitch Plinko」球數轉移圖建構器，靠 `split_field` 參數化 (投手版用 `bat_side`，打者版用 `pitch_hand`)，被 `compute_pitcher_statcast` 與 `compute_batter_statcast` 共用。
- `compute_pitch_movement_chart(pitches, max_points=700) -> dict`：投手球路移動散點圖資料，超過 `max_points` 時等間隔降採樣。

SHARED AGGREGATION

- `_spray_direction_from_location(p) -> Optional[str]`：座標缺失時用 `hit_location` 守位代碼 fallback 分類噴射方向。
- `_spray_direction_from_coordinates(p) -> Optional[str]`：主要分類路徑，用 Gameday 座標三角函數公式 (檔內有完整中文推導註解 + 出處)。
- `_compute_spray(in_play) -> dict`：{pull, straight, oppo, pull_air, spray_total}。
- `_aggregate_pitches(pitches) -> dict` (statcast.py:806-893)：核心單次遍歷分類函式，一次迴圈產出所有子清單 (swings, whiffs, called, in_zone, out_zone, in_zone_swings, out_zone_swings, in_zone_contact, in_play, bbe_ev, pa_final) 與計數 (gb, fb, ld, pu, barrels, hard_hits) + 噴射方向統計。是投手版、打者版、以及每個逐球種細項函式共用的核心。
- `_compute_woba(pa_final, woba_w) -> (woba_num, woba_den)`：排除故意四壞、犧牲觸擊、非 PA 事件。
- `_discipline_metrics(agg) -> dict`：swing_pct, whiff_pct, swstr_pct, csw_pct, z_swing_pct, o_swing_pct, z_contact_pct, zone_pct。
- `_batted_ball_metrics(agg, sport_level="") -> dict`：gb/ld/fb/pu/air_pct、pull/straight/oppo/pull_air_pct (僅噴射資料可用時才填)、barrel_pct, hard_hit_pct, avg_ev。△ **sport_level** 參數接收但函式體內完全沒用到 (已用 `grep` 確認呼叫端兩處都有傳值，但函式內無對應邏輯) —— 死參數。

PITCHER AGGREGATION

- `compute_pitcher_statcast(pitches, year=None, sport_level="") -> dict` (statcast.py:967-1001)：投手球季彙總入口。回傳 `total_pitches, pa_count, woba_against, hr_fb_pct, avg_extension, pitch_arsenal, pitch_outcomes, pitch_usage_by_count` (皆取自 `pitcher_bat_side_splits["all"]`)、`pitcher_bat_side_splits(all/L/R)`、`pitch_plinko`、`pitch_movement`，再 update 進 `discipline + batted-ball metrics`。△ **year** 參數接收但函式體內完全沒用到 —— 死參數 (已用 `grep` 確認 `sync.py:1195` 有傳 `year=yr`)。
- `_compute_pitch_arsenal_pitcher(pitches) -> list[dict]`：逐球種「我方武器庫」視角
—— `count/pct/velo/ivb/hb/spin/extension/v_rel/h_rel/zone_pct/chase_pct/whiff_pct/put_away_pct/two_strike_count/woba`。
- `_compute_pitch_outcomes_pitcher(pitches) -> list[dict]`：逐球種「對方打擊結果」視角
—— `strike_pct/z_whiff_pct/o_swing_pct/swstr_pct/csw_pct/put_away_pct/avg/woba/barrel_pct/hard_hit_pct`。
- `_compute_pitch_usage_by_count_pitcher(pitches) -> dict`：{pitch_types, rows}，依球數分桶 (前段/領先/落後/兩好球前/兩好球後) 的球種使用率。
- `_compute_pitcher_bat_side_splits(pitches) -> dict[str, dict]`：包裝上述三者，產出 all/L/R 三套。

BATTER AGGREGATION

- `compute_batter_statcast(pitches, year=None, sport_level="") -> dict` (statcast.py:1200-1247)：打者球季彙總入口。回傳 `total_pitches, pa_count, strike_pct, woba, max_ev, ev90` (90 百分位 EV，程式內註明樣本數 <10 顆 BBE 時數值會跟 TJStats 對不上)、`avg_la, swsp_pct, vs_pitch_types, pitch_plinko`，再 update `discipline + batted-ball`。同樣有 **year** 死參數問題。
- `_compute_vs_pitch_types_batter(pitches) -> list[dict]`：逐球種打者表現，排除 EP/FA (替補野手投球的假球種，與 TJStats/Baseball Savant 行為一致)，有真實球種時丟掉 UN 桶。

FIP / xWPCT

- `compute_fip(hr, bb, hbp, k, ip, sport_level, year, c_fip=None) -> Optional[float]`：標準 FIP 公式，常數查找鏈：精確 (level,year) → 同等級任一年 → 寫死 3.2 最終 fallback。
- `compute_xwpcct(fip, sport_level) -> Optional[float]`：Pythagpat 1.83 次方公式，LEAGUE_RA9 查無時 fallback 4.5。

PITCH-LOG DISPLAY

- `summarize_pitch_for_display(p) -> dict`：13 欄位的精簡投影，給 `builder.py` 寫出的每場 lazy-load JSON 檔用。

拆分相關觀察

內部區塊標題已經幾乎天然對應到模組邊界：`pitch_extraction.py` (EXTRACTION 段)、`pitch_classification.py` (CLASSIFICATION + SHARED AGGREGATION，因為 `_aggregate_pitches` 幾乎被所有東西依賴，建議和分類常數放一起)、`pitcher_statcast.py`、`batter_statcast.py`、`fip.py`。

三個值得在拆分前一併處理的小問題（不只是搬檔案，順手修正比較划算）：

1. `compute_pitcher_statcast / compute_batter_statcast` 的 `year` 參數、`_batted_ball_metrics` 的 `sport_level` 參數，目前都是「接了但函式體內沒用」的死參數——拆分前要決定是要接線（例如未來想用年份相關常數）還是直接刪掉，避免拆完之後死參數繼續在新檔案裡傳遞下去。
2. `_ratio` 的精度預設（這裡 `digits=3`，[builder.py](#) 那份寫死 `digits=4`）如果合併成一份共用函式，要先確認兩處用到的精度差異是刻意的還是巧合。
3. `get_woba_weights` 已是零呼叫端的死碼（wOBA 權重改成固定表後留下的舊相容介面），拆分時直接刪除即可，不需要搬到新檔案。

8. site_builder/wrc_plus.py — TJBat+ (wRC+) 計算（新增模組）

新增檔案：依 TJStats glossary 公式計算 wRC+（站上顯示為「TJBat+」）。是套件中唯一會在 build time 對外部網站（`tjstats.ca`，而非 `statsapi.mlb.com`）發 HTTP request 的模組，也是唯一直接依賴第三方 HTML 解析（`beautifulsoup4`，本次一併加進 `requirements.txt`）的模組。職責單一、檔案內無區塊標題也不需要，是新模組裡少見的「一次到位」案例。

常數

常數	說明
TIMEOUT = 15	對 tjstats.ca 請求的逾時秒數，與 <code>api.py.TIMEOUT</code> 同值但各自獨立定義（兩個模組沒有共用常數）
WOBA_SCALE = 1.24	TJStats wRC+ 公式裡把 wOBA 差值換算回 runs 的固定 scale
MIN_WRC_YEAR = 2021	早於此年份的球季不計算 wRC+（TJStats 資料涵蓋範圍限制）； <code>builder.py</code> 也直接 import 這個常數來判斷是否要插入「無 statcast 但有 wRC+」的摘要列
PF_LEVEL_PARAM	{tier key → tjstats.ca park-factors 頁面的 <code>pf_level</code> 查詢參數}，只涵蓋 MLB/AAA/AA/A+/A 五級（ <code>levels.py</code> 的 TIERS 中 A- / ROK / WIN / Minors 沒有對應頁面）
LC_LEVEL_CODE	{tier key → tjstats.ca league-constants 表格的 Level 代碼}，內容與 PF_LEVEL_PARAM 同一組等級，但拼法不同（ <code>hi_a / lo_a</code> vs <code>hi-a / lo-a</code> ）——同一份領域知識用兩個查找表存兩種拼法，是這個新模組自帶的小重複，與第 10 節列出的舊有跨檔案重複是同一類問題
WRC_LEVELS	<code>tuple(PF_LEVEL_PARAM.keys())</code> ，給 <code>builder.py</code> 判斷某筆 <code>season_stats</code> 是否屬於「可能有 wRC+」的等級

Functions

計算核心

- `compute_woba(stat) -> Optional[float]`：從 `season_stats` 列的計數型欄位（`ab/hits/doubles/triples/hr/hit_bb/ibb/hbp/sac_flies`）算 wOBA，公式與 `statcast.py` 的逐球版本（`_compute_woba`）同一套慣例（故意四壞球從分子分母都排除），但輸入資料源完全不同（這裡是球季累計數字，不是逐球資料）。權重直接引用 `statcast.WOBA_WEIGHTS`，是 `wrc_plus.py` 對 `statcast.py` 的唯一依賴。分母 `<=0` 回傳 `None`。
- `compute_wrc_plus(woba, pf_final, lg_woba, lg_r_pa) -> Optional[int]`：TJStats 公式 $100 \times (wRC/PA + PFm) / lg_R/PA$ ，四捨五入成整數。`lg_r_pa` 或 `pfm` 為 0 時回傳 `None`。

外部資料抓取（best-effort，失敗一律回傳 {} 不丟例外）

- `fetch_park_factors(level, year) -> dict[team_name, {pf_final, league}]`：抓 `tjstats.ca/park-factors/?pf_level=...&pf_season=...`，解析 `table.tjs-guts` 第一張表。
- `fetch_league_constants(year) -> dict[{level_code, league}, {lg_woba, lg_r_pa}]`：抓同一個 URL 但只帶 `lc_season` 參數，解析同頁面的第二張 `table.tjs-guts`（`park factors` 和 `league constants` 共用同一個 [tjstats.ca](#) 頁面，只是讀不同表格）。

Orchestration

- `annotate_wrc_plus(bundles) -> None`（`wrc_plus.py:166-229`）：`builder.py` 的唯一呼叫入口，在所有頁面渲染之前對 `_load_player_bundle` 產出的 `bundles`（`[(player, stats, logs), ...]`）原地修改。只處理非投手（`player.position != "P"`）。依（`year, sport_level`）分組後，組內 **PA 最多的那一筆**決定整組要用哪支球隊的 `park factor` / 聯盟（模擬 `TJStats` 對賽季中轉隊球員的處理方式），再對組內每一筆分別算 `wOBA -> wRC+`。MLB 列寫入新欄位 `wrc_plus_calc`（不覆寫 API 原生的 `wrc_plus`）；非 MLB 列直接寫入 `wrc_plus`（模板本來就會讀這個欄位）。內建 `pf_cache` / `lc_cache` 兩層快取，同一個（`level, year`）或 `year` 只會打一次 [tjstats.ca](#)。計算結果不寫回 **SQLite**，每次 `build` 都重新抓重新算。

拆分相關觀察

- 不需要拆，職責已經單一（一個外部資料源 + 一個計算公式 + 一個 `orchestration` 入口）。
- `PF_LEVEL_PARAM` / `LC_LEVEL_CODE` 兩個查找表內容同源但拼法不同，可考慮合併成一個 `{tier key -> (pf_param, lc_code)}` 的表減少維護成本，但影響範圍小，優先度低。
- 對外部網站做即時 HTML 解析（而非結構化 API）是套件中第一次出現的 `pattern`，`tjstats.ca` 改版會直接讓這個模組整段失效——值得在拆分/重構時考慮加一層更明確的「解析失敗」告警（目前只在 `print warning`，沒有跟 `_warn_orphaned_players` 之類既有的診斷輸出整合）。

9. site_builder/builder.py — SQLite → Jinja2 → HTML（職責最雜的大檔）

全套件「該拆」訊號最強的檔案：同時混雜了「跨等級統計合併運算」「SEO/結構化資料生成」「SQLite 資料載入」「Jinja 渲染 `orchestration`」四種彼此獨立的職責。

常數

常數	說明
<code>_PROJECT_ROOT</code>	專案根路徑
<code>_SITE_TITLE</code> , <code>_SITE_DESCRIPTION</code> , <code>_SITE_SAME_AS</code>	全站 SEO metadata
<code>_BAT_SIDE_SPLITS</code>	與 <code>statcast.py</code> 完全相同內容，重複定義
<code>_COUNT_USAGE_BUCKETS</code>	與 <code>statcast.py</code> 同語意但形狀不同（這裡只有（ <code>key, label, counts_label</code> ）三元組，沒有 <code>counts set</code> ，因為這裡是合併「已經算好的結果」而非從逐球資料重新分桶）
<code>_PLINKO_COUNTS</code>	與 <code>statcast.py</code> 完全相同內容，重複定義（字串形式 <code>"0-0"</code> vs statcast.py 的 <code>tuple</code> 形式 <code>(0,0)</code> ）
<code>_PLINKO_EDGES</code>	與 <code>statcast.py</code> 完全相同內容，重複定義

Functions

通用比率/分類（與 [statcast.py](#) 重複）

- `_ratio(num, den) -> Optional[float]`：固定 4 位小數。△ 與 `statcast._ratio`（預設 3 位）同概念不同簽名。
- `_is_unknown_pitch_type(pitch_type, pitch_name=None) -> bool`：△ 與 `statcast.py` 逐字相同實作。

跨等級 Statcast 合併（純運算，零渲染邏輯——全檔最大的「放錯位置」候選）

這 9 個函式（`builder.py:93-557`，約 280 行）處理的場景是：球員同一年在多個等級出賽（例如球季中升降），每個等級各有獨立的 `statcast` 數據，需要合併出一筆「合計」摘要列給卡片/詳細頁顯示。全程不涉及任何 **Jinja/HTML/檔案 I/O**，邏輯性質與 `statcast.py` 完全相同，只是輸入從「逐球資料」換成「已聚合的逐等級 `statcast dict`」。

- `_combine_pitch_type_data(entries, sc_key, rate_fields, include_pct=False) -> list[dict]`：泛用的「依球數加權平均」逐球種合併器，是下面三個函式的共用核心。`put_away_pct` 永遠用 `two_strike_count` 加權（特別處理）。
- `_combine_vs_pitch_types(entries) -> list[dict]`：打者視角包裝。
- `_combine_pitch_outcomes(entries) -> list[dict]`：投手「對方結果」視角包裝。

- `_combine_pitch_arsenal(entries) -> list[dict]`：投手「武器庫」視角包裝。
- `_combine_pitch_usage_by_count(entries) -> dict`：跨等級原始計數加總（非加權平均）的球數分桶使用率。
- `_combine_pitcher_bat_side_splits(entries) -> dict`：合併 all/L/R 三套分割（重新組裝 per-split entries 後再呼叫上面三個合併器）。
- `_combine_pitch_plinko(entries) -> dict`：跨等級原始計數加總的 Plinko 節點/邊。
- `_combine_pitch_movement(entries) -> dict`：合併逐等級球路移動散點，重新套用 `max_points=900` 降採樣。
- `_combine_statcast_dicts(entries) -> dict`（builder.py:478-556）：**頂層合併入口**。discipline/batted-ball/woba 欄位分三組分別用 `total_pitches / bbe / pa_count` 加權平均，`max_ev` 取跨等級最大值，再呼叫上述各個逐球種合併器組裝完整結果。產出球員跨等級時看到的「合計」列。

展示用字串工具

- `_pick_display_stat(stats_current, player) -> Optional[Obj]`：3 層優先序（球隊完全相符 → 目前等級相符 → 最高等級 fallback），決定卡片/英雄區要顯示哪一筆季資料。
- `_player_display_name(player) -> str`：`"{name_tw} {name_en}"` 或單純 `name_en`。
- `_player_canonical_path(player, is_retired=False) -> str`：URL 路徑片段。
- `_player_description(player) -> str`：meta description 文字產生器。

SEO / 結構化資料 (JSON-LD)

- `_index_structured_data(absolute_url, player_data) -> list[dict]`：WebSite + ItemList (schema.org)。
- `_player_structured_data(absolute_url, player, is_retired=False) -> list[dict]`：Person + BreadcrumbList。

檔案寫出

- `_write_robots(out_dir, sitemap_url)`：寫 robots.txt。
- `_write_sitemap(out_dir, urls)`：手刻 XML 字串寫 sitemap.xml（沒有用 XML library）。

資料載入

- `_load_player_bundle(cur, player_row) -> (player: Obj, stats: list[Obj], logs: list[Obj])`（builder.py:707-799）：載入單一球員的 `players + season_stats + game_logs` 完整資料，計算衍生欄位：`is_pitcher`, `birth_date(parse_date)`, `age`, `status_category/status_display`，每筆 stat 算 `level_order`，`latest_stat`, `available_years`, `latest_level_is_mlb`（依「最近一個有實際出賽的球季」而非「曾經到過 MLB」，驅動 headshot CDN 順序選擇）。對舊資料庫缺 `pitches_json` 欄位的情況做 feature-detection（LIMIT 0 查詢 try/except）。**iso**（打者 ISO 欄位）原本在這裡用 `slg - avg` 算一次，已移除——`helpers._compute_advanced_stats` 本來就會算同一個欄位（只在 None 時才填），這裡是重複計算，刪掉後邏輯只剩一份。

Orchestration (全檔最大函式)

- `build_static_site(db_path, year, output_dir, base_url="/", roster_file=None) -> None`（builder.py:802-1217，約 **415 行**）：單一入口做完全部事情：
 - i. 解析 `roster_ids` 篩選；重建輸出目錄（`shutil.rmtree + mkdir`），複製 `src/static`。
 - ii. 建 `jinja env`，設定 `build_time global (UTC+8)`。
 - iii. 開 `sqlite 連線`，驗證 `players table` 存在（不存在直接 `SystemExit`）。
 - iv. 依 `roster_ids` 載入所有球員列，警告 `orphan`（DB 有但 `roster.json` 沒有）。
 - v. 對每列呼叫 `_load_player_bundle`。
 - vi. **（新增）對全部 bundles 呼叫 `wrc_plus.annotate_wrc_plus(bundles)`**，在任何頁面渲染之前把 `wRC+/TJBat+` 原地寫進每位打者的 `season_stats` 列，讓後續 `active` 與 `retired` 兩條渲染路徑都能讀到同一份結果。
 - vii. 用 `is_active_player` 分流成 `active_bundles / retired_bundles`。
 - viii. 渲染 `index.html`（僅 `active`，依 `level_rank` 排序，含結構化資料）。
 - ix. 渲染 `retired/index.html`（`career-combined` 統計，依最高等級再依最近出賽日排序）。
 - x. ****對每一位球員（active+retired 都跑）**渲染 `player_detail.j2`：**`annotate_computed_stats + compute_year_groups` → 算 `mlb/mlb/total career` → 驗證並格式化 `next_game` 快照 → 算當季顯示用統計 → 攤平所有球季的 `fielding` 資料 → **把逐場 pitch log 寫成獨立 JSON 檔到 `out_dir/data/pitchlogs/{mlb_id}/{game_id}.json`**（lazy-load，避免球員 HTML 過大）→ 組 `statcast_by_year`（跨等級時呼叫 `_combine_statcast_dicts` 插入 `"_combined"` 合計列；對沒有 `statcast` 資料但有算出 `wRC+` 的打者季資料，補插入一筆 `sc={}` 的近空白項目，避免該年份因為沒有逐球資料而從 `Statcast Overview` 區塊整個消失）→ 組裝 ~25 個 key 的 `template context` → 渲染並寫出 `player/{id}/index.html` 或 `retired/player/{id}/index.html`。
 - xi. 寫 `404.html`。
 - xii. 組裝並寫 `sitemap.xml + robots.txt`。
 - xiii. 寫 `.nojekyll`。

此輪簡化：先前這裡會自行重新走訪所有 `game_logs.pitches_json`，依 `(year, sport_level)` 重新分組呼叫 `statcast.compute_pitch_movement_chart` 算出 `movement_by_year_level`，再決定要不要覆寫 `sc["pitch_movement"]`。這段邏輯已整段移除——`pitch_movement` 現在完全由 `statcast.compute_pitcher_statcast`（`statcast` 同步 pipeline）依等級算好直接存在每筆 `season_stats` 的 `statcast` dict 裡，`builder.py` 只需要 `sc.setdefault("pitch_movement", {})` 給沒有這個欄位的舊資料補一個空殼即可。對應地，`builder.py` 不再從 `site_builder.statcast import compute_pitch_movement_chart`。

拆分相關觀察

這個檔案至少混雜 4 種各自獨立成立的職責：

1. **跨等級 statcast 合併運算**（≈280 行，零渲染依賴）——邏輯性質與 `statcast.py` 相同，建議搬到 `statcast.py` 旁邊（例如獨立 `statcast_combine.py`），而不是留在「渲染」模組裡。
2. **SQLite → Obj 資料載入**（`_load_player_bundle + build_static_site` 內的球員查詢/orphan 警告區塊）。
3. **SEO / 結構化資料 / sitemap / robots**——一個自成一格、與統計合併邏輯零依賴的「站台 metadata」職責。
4. **Jinja 渲染 orchestration 本身**（決定每個頁面拿到什麼 context）。

即使不跨檔案搬動，`build_static_site` 單一函式做完 `index+retired+逐球員+404+sitemap` 全部流程、中間沒有函式邊界，光是拆成 `_build_index_page(...)`、`_build_retired_page(...)`、`_build_player_page(player, stats, logs, env, out_dir, ...)`、`_build_discovery_files(...)` 幾個內部函式，就能讓 `orchestration` 本身變得可讀。第 10 步（逐球員迴圈內容）本身份量已經足夠獨立成 `_build_player_context(player, all_stats, all_logs, year, out_dir, normalized_base_url) -> dict`，與「把 context 寫成檔案」這個動作分開。

`wrc_plus.annotate_wrc_plus(bundles)` 這次新增的呼叫本身是個好示範：`builder.py` 只負責「在對的時間點呼叫一次」，實際的外部資料抓取與計算公式全部留在 `wrc_plus.py`——新功能沒有重蹈「把領域邏輯寫進渲染模組」的覆轍，值得當作之後搬移跨等級合併區塊時的參照範例。

10. 跨檔案重複（直接影響拆分順序的關鍵發現）

項目	statcast.py	builder.py	備註
<code>_BAT_SIDE_SPLITS</code>	有	有	內容逐字相同
<code>_COUNT_USAGE_BUCKETS</code>	有 (dict, 含 <code>counts</code> set)	有 (tuple, 無 <code>counts</code>)	同一份領域知識，形狀已經開始分歧——如果哪天改了分桶的 label，兩邊很容易改到只剩一邊，互相對不上
<code>_PLINKO_COUNTS</code>	有 (tuple of int-tuple)	有 (tuple of string)	內容邏輯相同，型別表示法不同
<code>_PLINKO_EDGES</code>	有	有	內容逐字相同
<code>_is_unknown_pitch_type</code>	有	有	byte-for-byte 相同實作
<code>_ratio</code>	<code>digits=3</code> （預設）	<code>digits=4</code> （寫死）	同樣用途、精度不同——合併前要先確認兩邊精度差異是否刻意

判斷依據：這 6 組重複全部圍繞「球種/球數分桶」這個領域概念，且都是 `builder.py` 在做「跨等級合併」時，需要重新使用 `statcast.py` 計算單一等級資料時用過的同一套常數/邏輯。最自然的修法是把這些常數與 `_is_unknown_pitch_type / _ratio` 留在 `statcast.py`（或拆出的子模組）作為唯一定義，`builder.py`（或拆出後接手「跨等級合併」職責的新模組）改成 `from .statcast import ...`。這也呼應第 9 節的結論：**`builder.py` 裡的「跨等級 statcast 合併」這 9 個函式，本質上屬於 `statcast.py` 的領域，不屬於「渲染」領域**——把它們搬過去，重複的常數會自然消失，不需要額外建一個共用常數檔。

其他跨檔案觀察：

- **Obj 資料載體**：`helpers.py` 定義，`builder.py` 全面使用（`_load_player_bundle` 把 SQLite 列轉成 Obj），但 `sync.py` 完全不用（內部用原生 dict）。拆分 `helpers.py` 時，Obj 應該跟著「統計聚合公式」那組一起搬，因為 `compute_career / compute_season_combined / _aggregate_stats` 等函式回傳值都是 Obj。
- **helpers.py 對 levels.py 的 re-export**：`sync.py`、`builder.py` 都還在用 `from site_builder.helpers import (level_rank, ...)` 而非直接 `from .levels import`（已用 `grep` 確認）。這代表「等級邏輯曾經搬過一次家，但呼叫端沒有全部跟著改」——是個活生生的「半途而廢的重構」案例，提醒這次拆分要嘛把舊路徑徹底清乾淨、嘛保留轉接層但要意識到它的存在成本。
- **MAX_WORKERS**：`sync.py` 單一常數被「球員 profile 抓取」與「比賽 live-feed 抓取」兩種不同性質的平行工作共用，拆成兩個 pipeline 檔案時，這個共用值得重新評估是否該分開調校。

11. 給拆分決策的速查重點

依「該優先處理」排序：

1. **builder.py** 的跨等級合併區塊（9 個函式， ≈ 280 行）→ 搬去和 **statcast.py** 同一個職責域。這一步同時解決第 10 節列出的 5 組常數/函式重複，是 ROI 最高的第一刀。
2. **helpers.py** 的 `_compute_advanced_stats`（ ≈ 215 行）依「打者公式 / 投手公式」現成的內部分段拆開，是套件中最大、職責最不單一的單一函式。
3. **sync.py** 拆成三塊：DB schema/migrations、Pipeline A（profile/season 同步）、Pipeline B（statcast 同步）——兩條 pipeline 目前共用一個檔案但業務邏輯零交集。
4. **builder.py** 的 `build_static_site`（ ≈ 415 行單一函式）依頁面類型拆成內部子函式（index / retired / 單一球員 / discovery files），即使最終不跨檔案搬動也值得做。
5. **helpers.py** 整體依 5 種職責（utils / roster 狀態 / 單位轉換 / 統計聚合 / obj）拆分，順手把已經不再對齊的 `levels.py` re-export 路徑清乾淨。
6. **statcast.py** 依現有區塊標題拆成子模組（extraction / classification+aggregation / pitcher / batter / fip），優先序較低是因為內部已經分得很清楚，拆不拆對可讀性影響相對小。

不建議動的：`levels.py`（已是最佳實踐範本）、`jinja_env.py`（邊界已清楚）、`api.py`（職責單一，只需小修錯誤處理不對稱與 timeout 字面量不一致兩個小問題）、`wrc_plus.py`（新增模組，職責已經單一，唯一的小重複——PF_LEVEL_PARAM / LC_LEVEL_CODE 兩個拼法不同的查找表——影響範圍小，不急）。